

Spring Security - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

SPRING SECURITY — MASTER NOTES	6
A Complete Reference Book for Interviews, Production Work & System Understanding	6
Includes full, step-by-step integration guides for JWT, OAuth2 (Google & GitHub Sign-In), Refresh Tokens, CORS, and CSRF	6
TOPIC 1: AUTHENTICATION	6
1. Introduction	6
1.1 What is it?	6
1.2 Why was it introduced	6
2. Setup	6
3. The Security Filter Chain — Internal Working	7
4. Basic Configuration — SecurityFilterChain	7
5. Real Life Analogy	7
6. Frequently Asked Interview Questions	7
TOPIC 2: AUTHORIZATION	8
1. Introduction	8
2. URL-Based Authorization	8
3. Method-Level Authorization (More Granular, AOP-Based — Ties to Core Java's AOP Topic)	8
4. Real Life Analogy	9
5. Frequently Asked Interview Questions	9
TOPIC 3: JWT (JSON WEB TOKEN) — FULL STEP-BY-STEP INTEGRATION	9
1. Introduction	9
1.1 What is it, and why?	9
1.2 JWT Structure	10
2. FULL STEP-BY-STEP: Implementing JWT Authentication	10
STEP 1 — Add Dependencies	10
STEP 2 — Configure a Secret Key	10
STEP 3 — Build a JwtUtil Class (Generate & Validate Tokens)	11
STEP 4 — Build a Custom JwtAuthenticationFilter (Intercepts EVERY Request)	12
STEP 5 — Build a UserDetailsService (Loads User Data)	13

STEP 6 — Wire Everything Together in SecurityConfig	13
STEP 7 — Build the Login Endpoint	14
STEP 8 — The CLIENT Uses the Token on Every Subsequent Request	14
3. The Full Request Flow Diagram	14
4. Real Life Analogy	14
5. Edge Cases	15
6. Common Mistakes	15
7. Best Practices	15
8. Frequently Asked Interview Questions	15

TOPIC 4: OAUTH2 — FULL STEP-BY-STEP "SIGN IN WITH GOOGLE" & "SIGN IN WITH GITHUB" INTEGRATION **16**

1. Introduction	16
1.1 What is it, and why?	16
1.2 Why this matters	16
2. Key OAuth2 Roles and Terminology	16
3. The Full OAuth2 "Authorization Code" Flow (What Happens Conceptually)	17
4. FULL STEPS: Setting Up "Sign In With Google"	17
STEP 1 — Create a Google Cloud Project and OAuth Credentials	17
STEP 2 — Add the Dependency	17
STEP 3 — Configure Google in application.yml	18
STEP 4 — Enable OAuth2 Login in SecurityConfig	18
STEP 5 — Handle a Successful Login (Create/Find a Local User, Issue YOUR OWN JWT)	19
STEP 6 — Trigger the Flow From Your Frontend	19
5. FULL STEPS: Setting Up "Sign In With GitHub" (Same App, Second Provider)	19
STEP 1 — Register an OAuth App on GitHub	19
STEP 2 — Add GitHub Configuration to application.yml (Alongside Google)	20
STEP 3 — Handle GitHub's Slightly Different User Attributes	20
STEP 4 — Trigger the GitHub Flow From Your Frontend	21
6. Real Life Analogy	21
7. Edge Cases	21
8. Common Mistakes	21
9. Best Practices	21
10. Frequently Asked Interview Questions	21

TOPIC 5: ROLE-BASED ACCESS CONTROL (RBAC)	22
1. Introduction	22
2. Modeling Roles in Your Entity	22
3. Enforcing Roles — URL and Method Level	22
4. Real Life Analogy	23
5. Frequently Asked Interview Questions	23
TOPIC 6: PASSWORD ENCODING	23
1. Introduction	23
2. Why BCrypt (Not Plain SHA-256/MD5)	24
3. Code Example	24
4. Real Life Analogy	24
5. Common Mistakes	25
6. Frequently Asked Interview Questions	25
TOPIC 7: REFRESH TOKENS — FULL FLOW	25
1. Introduction	25
2. Why Both an Access Token AND a Refresh Token	25
3. FULL STEP-BY-STEP Implementation	26
STEP 1 — Store Refresh Tokens (Database-Backed, So They Can Be Revoked)	26
STEP 2 — Generate a Refresh Token on Login	26
STEP 3 — Return BOTH Tokens at Login	26
STEP 4 — Build a /refresh Endpoint	27
STEP 5 — Client-Side Flow	27
4. Revoking Refresh Tokens (Logout)	27
5. Real Life Analogy	27
6. Frequently Asked Interview Questions	27
TOPIC 8: CORS (CROSS-ORIGIN RESOURCE SHARING) — FULL CONFIGURATION	28
1. Introduction	28
2. Why the Browser Blocks This by Default (The "Same-Origin Policy")	28
3. FULL STEP-BY-STEP Configuration	29
STEP 1 — Configure CORS Globally in SecurityConfig	29

STEP 2 — (Alternative) Simple, Controller-Level CORS (For Quick/Small Projects)	29
4. The CORS Preflight Request — Internal Working	29
5. Real Life Analogy	29
6. Edge Cases	30
7. Common Mistakes	30
8. Frequently Asked Interview Questions	30

TOPIC 9: CSRF (CROSS-SITE REQUEST FORGERY) 30

1. Introduction	30
2. How the Attack Works	31
3. How Spring Security's CSRF Protection Works	31
4. Code Example — CSRF in a Traditional Server-Rendered App (Enabled, Default Behavior) . . .	31
5. Why JWT-Based Stateless APIs Typically DISABLE CSRF Protection	31
6. Real Life Analogy	31
7. Edge Cases	32
8. Common Mistakes	32
9. Frequently Asked Interview Questions	32
10. Revision Notes (Entire Spring Security Section Consolidated)	33
11. Homework (Covers Entire Spring Security Section)	33
FINAL SUMMARY — SPRING SECURITY COMPLETE	33

SPRING SECURITY — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

Includes full, step-by-step integration guides for JWT, OAuth2 (Google & GitHub Sign-In), Refresh Tokens, CORS, and CSRF

TOPIC 1: AUTHENTICATION

1. Introduction

1.1 What is it?

Authentication answers the question **"WHO are you?"** — verifying a user's claimed identity (typically via username/password, a token, or a third-party login like Google). Spring Security is THE standard framework for handling this (and Authorization, next topic) in Spring Boot applications.

1.2 Why was it introduced

Every real application handling user data needs to answer "who is making this request" BEFORE deciding what they're allowed to do. Building this from scratch (parsing credentials, hashing passwords correctly, protecting against common attacks like session fixation/CSRF) is genuinely hard to get right — Spring Security provides a battle-tested, standard, deeply-integrated solution instead of everyone reinventing (and likely under-securing) this themselves.

2. Setup

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

The MOMENT you add this dependency, Spring Boot AUTOMATICALLY secures EVERY endpoint by default, generating a random password printed to your console at startup, and requiring HTTP Basic Auth login for EVERYTHING — a deliberate "secure by default" design.

3. The Security Filter Chain — Internal Working

```
Incoming HTTP Request
|
v
+-----+
| SECURITY FILTER CHAIN |
| (a series of Servlet Filters - EXACTLY the mechanism from |
| Advanced Java's Filters topic!) |
| |
| 1. Extract credentials (username/password, JWT token, etc.) |
| 2. Pass to an AuthenticationManager |
| 3. AuthenticationManager delegates to an AuthenticationProvider |
| 4. AuthenticationProvider verifies credentials (checks password, |
| validates JWT signature, etc.) |
| 5. IF valid: populate the SecurityContext with an Authentication |
| object (WHO the user is, WHAT authorities/roles they have) |
| 6. IF invalid: reject with 401 Unauthorized |
+-----+
|
v
Request proceeds to your Controller (if authenticated + authorized)
```

4. Basic Configuration — `SecurityFilterChain`

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll() // NO authentication needed
                .requestMatchers("/api/admin/**").hasRole("ADMIN") // must have ADMIN role
                .anyRequest().authenticated() // everything else needs SOME login
            )
            .httpBasic(Customizer.withDefaults()); // simple HTTP Basic Auth for now

        return http.build();
    }
}
```

5. Real Life Analogy

- **Bank:** Authentication is like a bank's ID VERIFICATION step at the entrance — checking that you actually ARE who your ID card claims. It does NOT yet decide what you're allowed to DO inside (that's Authorization, next topic) — just confirms your IDENTITY first.

6. Frequently Asked Interview Questions

- 1 **What does Authentication answer?** "WHO are you?" — verifying a claimed identity.

- 2 **What happens the MOMENT you add `spring-boot-starter-security` with no further configuration?** Every endpoint is secured by default, requiring login via HTTP Basic Auth, with a randomly-generated password printed to the console.
 - 3 **What internal mechanism does Spring Security use to intercept requests?** A chain of Servlet Filters (the Security Filter Chain), the exact same underlying Filter mechanism covered in Advanced Java.
 - 4 **What object holds the authenticated user's identity/authorities after successful login?** The `Authentication` object, stored in the `SecurityContext`.
-

TOPIC 2: AUTHORIZATION

1. Introduction

What is it? Authorization answers "WHAT are you allowed to do?" — determining, AFTER a user's identity is verified (Authentication), whether they have PERMISSION to access a specific resource or perform a specific action.

2. URL-Based Authorization

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/public/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/employees/**").hasAnyRole("USER", "ADMIN")
    .requestMatchers(HttpMethod.DELETE, "/api/employees/**").hasRole("ADMIN") // ONLY admins can delete
    .anyRequest().authenticated()
);
```

3. Method-Level Authorization (More Granular, AOP-Based — Ties to Core Java's AOP Topic)

```
@Configuration
@EnableMethodSecurity // MUST be enabled for @PreAuthorize/@PostAuthorize to work
public class SecurityConfig { }

@Service
public class EmployeeService {

    @PreAuthorize("hasRole('ADMIN')") // checked BEFORE the method runs
    public void deleteEmployee(Long id) {
        // ...
    }

    @PreAuthorize("#employeeId == authentication.principal.id or hasRole('ADMIN')")
    public Employee getOwnProfile(Long employeeId) { // users can view THEIR OWN data, or admins can view ANY
        return employeeRepository.findById(employeeId).orElseThrow();
    }

    @PostAuthorize("returnObject.ownerId == authentication.principal.id")
    public Document getDocument(Long id) { // checked AFTER the method runs, using the RETURN VALUE
        return documentRepository.findById(id).orElseThrow();
    }
}
```



```
}  
}
```

`@PreAuthorize/@PostAuthorize` are IMPLEMENTED using Spring AOP (Core Java's Spring Core AOP topic) — proxying the method call to check authorization BEFORE (or AFTER, using the return value) the actual business logic executes.

4. Real Life Analogy

- **Bank:** Authorization is like the bank, HAVING ALREADY confirmed your identity, now checking WHAT you're allowed to do — a regular customer (a `USER` role) can view/withdraw from THEIR OWN account, but only a bank `MANAGER` (`ADMIN` role) can approve a large loan or close ANY account.

5. Frequently Asked Interview Questions

- 1 **What's the difference between Authentication and Authorization?** Authentication verifies WHO you are; Authorization determines WHAT you're allowed to do, AFTER identity is confirmed.
- 2 **What annotation enables method-level security checks like `@PreAuthorize`?** `@EnableMethodSecurity`.
- 3 **What's the difference between `@PreAuthorize` and `@PostAuthorize`?** `@PreAuthorize` checks BEFORE the method executes; `@PostAuthorize` checks AFTER, allowing the check to reference the method's RETURN VALUE.
- 4 **What underlying mechanism implements `@PreAuthorize`?** Spring AOP — the method call is intercepted via a proxy, and the authorization check runs before the actual method body.

TOPIC 3: JWT (JSON WEB TOKEN) — FULL STEP-BY-STEP INTEGRATION

1. Introduction

1.1 What is it, and why?

Traditional session-based authentication (Advanced Java's Session Management topic) requires the SERVER to remember every logged-in user's session in memory — problematic for STATELESS, horizontally-scaled REST APIs (recall the multi-server session-sharing problem from Advanced Java Topic 4). **JWT (JSON Web Token)** solves this by putting the user's identity/claims INSIDE a cryptographically-SIGNED token that the CLIENT holds and sends with every request — the server verifies the signature and needs to remember NOTHING between requests.

1.2 JWT Structure

HEADER.PAYLOAD.SIGNATURE

eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbmGljZSI6ImV4cCI6MTcwMDAwMDAwMH0.4f8a2b1c...

HEADER: {"alg": "HS256", "typ": "JWT"} (base64-encoded)

PAYLOAD: {"sub": "alice", "role": "ADMIN", "exp": 1700000000} (base64-encoded - NOT encrypted, just encoded!)

SIGNATURE: HMACSHA256(base64(header) + "." + base64(payload), SECRET_KEY)

Critical fact: The payload is only BASE64-ENCODED, NOT encrypted — anyone can DECODE and read it (never put passwords/sensitive secrets inside a JWT payload). The SIGNATURE is what prevents TAMPERING — if even one character of the header/payload changes, the signature verification fails.

2. FULL STEP-BY-STEP: Implementing JWT Authentication

STEP 1 — Add Dependencies

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-api</artifactId>
<version>0.12.5</version>
</dependency>
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-impl</artifactId>
<version>0.12.5</version>
<scope>runtime</scope>
</dependency>
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-jackson</artifactId>
<version>0.12.5</version>
<scope>runtime</scope>
</dependency>
```

STEP 2 — Configure a Secret Key

```
# application.properties
jwt.secret=ThisIsAVeryLongSecretKeyThatShouldBeAtLeast256BitsForHS256Algorithm
jwt.expiration-ms=3600000
```

NEVER **hardcode** this directly in source code committed to Git — use an environment variable in real production (same principle as the Configuration Properties topic in Spring Boot).

STEP 3 — Build a `JwtUtil` Class (Generate & Validate Tokens)

```
package com.jobsradar.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import javax.crypto.SecretKey;
import java.util.Date;
import java.util.function.Function;

@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration-ms}")
    private long expirationMs;

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(secret.getBytes());
    }

    public String generateToken(String username, String role) {
        return Jwts.builder()
            .subject(username)
            .claim("role", role)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(getSigningKey())
            .compact();
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public <T> T extractClaim(String token, Function<Claims, T> resolver) {
        Claims claims = Jwts.parser().verifyWith(getSigningKey()).build()
            .parseSignedClaims(token).getPayload();
        return resolver.apply(claims);
    }

    public boolean isTokenValid(String token, String username) {
        String extractedUsername = extractUsername(token);
        return extractedUsername.equals(username) && !isTokenExpired(token);
    }

    private boolean isTokenExpired(String token) {
        Date expiration = extractClaim(token, Claims::getExpiration);
        return expiration.before(new Date());
    }
}
```

STEP 4 — Build a Custom `JwtAuthenticationFilter` (Intercepts EVERY Request)

```
package com.jobsradar.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter { // runs EXACTLY once per request

    private final JwtUtil jwtUtil;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtUtil jwtUtil, UserDetailsService userDetailsService) {
        this.jwtUtil = jwtUtil;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain
    chain)
    throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization"); // expects: "Bearer <token>"

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            chain.doFilter(request, response); // no token - let it pass through (may still be rejected later
            return; // by authorizeHttpRequests rules, if the endpoint needs auth)
        }

        String jwt = authHeader.substring(7); // strip the "Bearer " prefix
        String username = jwtUtil.extractUsername(jwt);

        if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails = userDetailsService.loadUserByUsername(username);

            if (jwtUtil.isTokenValid(jwt, username)) {
                UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                    userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authToken); // MARKS the request as
                authenticated!
            }
        }
        chain.doFilter(request, response); // continue the filter chain (Advanced Java's Filters mechanism!)
    }
}
```

STEP 5 — Build a UserDetailsService (Loads User Data)

```
@Service
public class CustomUserDetailsService implements UserDetailsService {
    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) { this.userRepository =
userRepository; }

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found: " + username));

        return org.springframework.security.core.userdetails.User
            .withUsername(user.getUsername())
            .password(user.getPassword()) // ALREADY BCrypt-hashed (Topic 6)
            .authorities("ROLE_" + user.getRole())
            .build();
    }
}
```

STEP 6 — Wire Everything Together in SecurityConfig

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter) { this.jwtAuthFilter = jwtAuthFilter; }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // stateless APIs don't need CSRF (Topic 9)
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // NO server-side
sessions!
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll() // login/register - public
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class); // insert OUR filter
into the chain

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws
Exception {
        return config.getAuthenticationManager();
    }
}
```

STEP 7 — Build the Login Endpoint

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final JwtUtil jwtUtil;
    private final UserRepository userRepository;

    public AuthController(AuthenticationManager authenticationManager, JwtUtil jwtUtil, UserRepository
userRepository) {
        this.authenticationManager = authenticationManager;
        this.jwtUtil = jwtUtil;
        this.userRepository = userRepository;
    }

    @PostMapping("/login")
    public ResponseEntity<Map<String, String>> login(@RequestBody LoginRequest request) {
        authenticationManager.authenticate( // throws BadCredentialsException if wrong
new UsernamePasswordAuthenticationToken(request.username(), request.password()));

        User user = userRepository.findByUsername(request.username()).orElseThrow();
        String token = jwtUtil.generateToken(user.getUsername(), user.getRole());

        return ResponseEntity.ok(Map.of("token", token));
    }
}

record LoginRequest(String username, String password) {}
```

STEP 8 — The CLIENT Uses the Token on Every Subsequent Request

```
# Client stores the token from login, then sends it on EVERY protected request:
curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGljZSJ9..." \
http://localhost:8080/api/employees
```

3. The Full Request Flow Diagram

```
[Client] [Server]
| |
| |
|--- POST /api/auth/login ----->|
| {username, password} | AuthenticationManager verifies credentials (BCrypt check)
|<--- 200 OK {token: "eyJ..."} ----| JwtUtil generates a signed JWT
| |
| [Client STORES the token] |
| |
|--- GET /api/employees ----->|
| Header: Authorization: | JwtAuthenticationFilter intercepts, extracts token,
| Bearer eyJ... | verifies signature + expiration, sets SecurityContext
|<--- 200 OK [employee data] -----| Controller executes normally, now knowing WHO the user is
```

4. Real Life Analogy

- **Restaurant:** A JWT is like a SIGNED, TAMPER-EVIDENT wristband a restaurant gives you at check-in (login) — it contains your table number and meal-plan tier written on it (the payload), and a special hologram (signature) proving the RESTAURANT itself issued it. Any staff member can quickly VERIFY the hologram is genuine and read your details directly off the wristband, WITHOUT needing to walk back to a central reception desk (a server-side session store) to look you up every single time.

5. Edge Cases

Case	Result
An expired JWT sent with a request	<code>isTokenExpired()</code> check fails, <code>SecurityContext</code> is never set, the request is rejected as unauthenticated
A TAMPERED JWT (payload manually edited)	Signature verification FAILS — <code>Jwts.parser()</code> throws a <code>SignatureException</code>
Storing a JWT in <code>localStorage</code> (a common but debated practice)	Vulnerable to XSS attacks (malicious JavaScript could read it); storing in an <code>HttpOnly</code> cookie is often considered safer (ties back to the Cookies topic in Advanced Java)

6. Common Mistakes

- Putting sensitive data (passwords, secrets) directly in the JWT payload — it's only ENCODED, not encrypted, and trivially readable by anyone who intercepts it.
- Using a WEAK or hardcoded, committed-to-Git secret key for signing — allows an attacker to potentially forge valid tokens.
- Not setting a REASONABLE expiration time — tokens that never expire are a serious security risk if ever leaked.

7. Best Practices

- Keep JWT payloads MINIMAL (just user ID/username and roles) — never sensitive data.
- Use a STRONG, environment-variable-sourced secret key (never hardcoded/committed).
- Set a SHORT expiration time (e.g., 15-60 minutes) and use Refresh Tokens (Topic 7) for renewing access without re-login.

8. Frequently Asked Interview Questions

- 1 **What is a JWT, and what are its three parts?** A JSON Web Token — a self-contained, signed token with three parts: Header, Payload, Signature.
- 2 **Is a JWT's payload encrypted?** No — only base64-ENCODED, fully readable by anyone; the signature only prevents TAMPERING, not reading.
- 3 **How does JWT solve the multi-server session-sharing problem from traditional session-based auth?** The token itself carries the user's identity/claims, verified via signature — the server needs NO shared session store, since each request is self-contained and independently verifiable.
- 4 **What Spring Security component intercepts every request to check for a JWT?** A custom `OncePerRequestFilter` (like `JwtAuthenticationFilter`), inserted into the Security Filter Chain via `addFilterBefore()`.
- 5 **Why should JWTs have a short expiration time?** To limit the damage if a token is ever leaked/stolen — combined with refresh tokens, this balances security with user convenience.

TOPIC 4: OAUTH2 — FULL STEP-BY-STEP "SIGN IN WITH GOOGLE" & "SIGN IN WITH GITHUB" INTEGRATION

1. Introduction

1.1 What is it, and why?

OAuth2 is an INDUSTRY-STANDARD protocol that lets your application let users log in using an EXISTING account from a trusted third party (Google, GitHub, Facebook) — WITHOUT your application ever seeing or storing that user's actual Google/GitHub password. Instead, the third party (the "Authorization Server") verifies the user directly and hands YOUR application a TOKEN proving who they are.

1.2 Why this matters

- **User convenience:** no new password to create/remember for your app.
- **Security:** your application NEVER handles the user's actual Google/GitHub password — eliminating an entire class of risk (you can't leak a password you never had).
- **Trust:** users are often more comfortable logging in via a provider they already trust than creating yet another account.

2. Key OAuth2 Roles and Terminology

Term	Meaning
Resource Owner	The USER, who owns their Google/GitHub account
Client	YOUR application, requesting access
Authorization Server	Google's/GitHub's own login server, which authenticates the user and issues tokens
Resource Server	The server holding the actual protected user data (often the SAME as the Authorization Server for Google/GitHub)
Authorization Code	A short-lived code the Authorization Server gives your app after the user logs in, EXCHANGED for an actual access token
Access Token	The token your app uses to fetch the user's basic profile info from Google/GitHub

3. The Full OAuth2 "Authorization Code" Flow (What Happens Conceptually)

1. User clicks "Sign in with Google" on YOUR app
2. YOUR APP redirects the user's browser to GOOGLE'S login page
3. User logs into GOOGLE (enters their Google password - DIRECTLY on Google's own page, never yours!)
4. Google asks the user: "Allow [Your App] to access your basic profile info?"
5. User clicks "Allow"
6. Google redirects the user's browser BACK to YOUR app, with a temporary AUTHORIZATION CODE attached
7. YOUR APP'S BACKEND exchanges that code (server-to-server, with your Client Secret) for an ACCESS TOKEN
8. YOUR APP uses that access token to fetch the user's Google profile (name, email)
9. YOUR APP creates/finds a LOCAL user account matching that Google identity, and logs them in (typically by issuing YOUR OWN JWT, from Topic 3, for subsequent requests)

4. FULL STEPS: Setting Up "Sign In With Google"

STEP 1 — Create a Google Cloud Project and OAuth Credentials

- 1 Go to <https://console.cloud.google.com/> and log in with any Google account.
- 2 Click the project dropdown at the top → **"New Project"** → give it a name (e.g., "MyApp Auth") → **Create**.
- 3 In the left sidebar, navigate to **APIs & Services** → **OAuth consent screen**.
 - Choose **User Type: External** (unless you have a Google Workspace organization) → **Create**.
 - Fill in: App name, User support email, Developer contact email → **Save and Continue**.
 - On the "Scopes" step, add the scopes `.../auth/userinfo.email` and `.../auth/userinfo.profile` → **Save and Continue**.
 - On "Test users" (while your app is unpublished), add your own Google email as a test user → **Save and Continue**.
- 4 Navigate to **APIs & Services** → **Credentials** → click **"+ CREATE CREDENTIALS"** → **OAuth client ID**.
 - Application type: **Web application**.
 - Name: "MyApp Web Client".
 - Under **Authorized redirect URIs**, add:
`http://localhost:8080/login/oauth2/code/google`
(This EXACT path is Spring Security's DEFAULT redirect URI pattern:
`/login/oauth2/code/{registrationId}`.)
 - Click **Create**.
- 5 Google now shows you a **Client ID** and **Client Secret** — copy BOTH; you'll need them in Step 3 below.

STEP 2 — Add the Dependency

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

STEP 3 — Configure Google in `application.yml`

```
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: YOUR_GOOGLE_CLIENT_ID_HERE
            client-secret: YOUR_GOOGLE_CLIENT_SECRET_HERE
        scope:
          - email
          - profile
```

Spring Boot ALREADY knows Google's OAuth2 endpoints internally (it's a pre-configured "common provider") — you only need to supply your `client-id` and `client-secret`, nothing else for Google specifically.

STEP 4 — Enable OAuth2 Login in `SecurityConfig`

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/login/**", "/api/public/**").permitAll()
            .anyRequest().authenticated()
        )
        .oauth2Login(oauth2 -> oauth2
            .successHandler(oAuth2LoginSuccessHandler) // OUR custom handler - Step 5 below
        );
    return http.build();
}
```

STEP 5 — Handle a Successful Login (Create/Find a Local User, Issue YOUR OWN JWT)

```
@Component
public class OAuth2LoginSuccessHandler implements AuthenticationSuccessHandler {

    private final UserRepository userRepository;
    private final JwtUtil jwtUtil;

    public OAuth2LoginSuccessHandler(UserRepository userRepository, JwtUtil jwtUtil) {
        this.userRepository = userRepository;
        this.jwtUtil = jwtUtil;
    }

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request, HttpServletResponse response,
        Authentication authentication) throws IOException {
        OAuth2User oAuthUser = (OAuth2User) authentication.getPrincipal();

        String email = oAuthUser.getAttribute("email");
        String name = oAuthUser.getAttribute("name");

        User user = userRepository.findByUsername(email)
            .orElseGet(() -> {
                User newUser = new User();
                newUser.setUsername(email);
                newUser.setFullName(name);
                newUser.setRole("USER");
                newUser.setPassword(""); // no local password - this user ONLY logs in via OAuth2
                return userRepository.save(newUser);
            });

        String jwt = jwtUtil.generateToken(user.getUsername(), user.getRole());

        // Redirect back to YOUR frontend, with YOUR OWN JWT attached (e.g., as a query param, or set as a
        // cookie)
        response.sendRedirect("http://localhost:3000/oauth-success?token=" + jwt);
    }
}
```

STEP 6 — Trigger the Flow From Your Frontend

```
<!-- Simply a link/button pointing to Spring Security's built-in OAuth2 login-initiation endpoint -->
<a href="http://localhost:8080/oauth2/authorization/google">Sign in with Google</a>
```

Spring Security AUTOMATICALLY provides this `/oauth2/authorization/{registrationId}` endpoint — clicking it kicks off the ENTIRE flow described in Section 3 automatically; you never manually build the Google redirect URL yourself.

5. FULL STEPS: Setting Up "Sign In With GitHub" (Same App, Second Provider)

STEP 1 — Register an OAuth App on GitHub

- 1 Go to <https://github.com/settings/developers> (while logged into GitHub).
- 2 Click "OAuth Apps" → "New OAuth App".

- 3 Fill in:
 - **Application name:** "MyApp".
 - **Homepage URL:** `http://localhost:8080` (or your real domain in production).
 - **Authorization callback URL:**
`http://localhost:8080/login/oauth2/code/github`
(Again, matching Spring Security's default `/login/oauth2/code/{registrationId}` pattern, this time for github.)
- 4 Click **"Register application"**.
- 5 On the resulting page, copy the **Client ID**, then click **"Generate a new client secret"** and copy that too.

STEP 2 — Add GitHub Configuration to `application.yml` (Alongside Google)

```
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: YOUR_GOOGLE_CLIENT_ID_HERE
            client-secret: YOUR_GOOGLE_CLIENT_SECRET_HERE
            scope: [email, profile]
          github:
            client-id: YOUR_GITHUB_CLIENT_ID_HERE
            client-secret: YOUR_GITHUB_CLIENT_SECRET_HERE
            scope: [read:user, user:email]
```

GitHub is ALSO a Spring Boot "common provider" (like Google) — same minimal configuration needed, no manual endpoint URLs required.

STEP 3 — Handle GitHub's Slightly Different User Attributes

```
@Override
public void onAuthenticationSuccess(HttpServletRequest request, HttpServletResponse response,
Authentication authentication) throws IOException {
    OAuth2User oauthUser = (OAuth2User) authentication.getPrincipal();
    OAuth2AuthenticationToken oauthToken = (OAuth2AuthenticationToken) authentication;
    String provider = oauthToken.getAuthorizedClientRegistrationId(); // "google" or "github"

    String email;
    String name;

    if ("github".equals(provider)) {
        email = oauthUser.getAttribute("email"); // GitHub may return null here if the user's email is
        private!
        name = oauthUser.getAttribute("name");
        if (email == null) {
            email = oauthUser.getAttribute("login") + "@users.noreply.github.com"; // a sensible fallback
        }
    } else { // "google"
        email = oauthUser.getAttribute("email");
        name = oauthUser.getAttribute("name");
    }

    // ... same "find or create local user, issue JWT" logic as the Google example above ...
}
```

STEP 4 — Trigger the GitHub Flow From Your Frontend

```
<a href="http://localhost:8080/oauth2/authorization/github">Sign in with GitHub</a>
```

6. Real Life Analogy

- **Bank:** OAuth2 is like a bank accepting a GOVERNMENT-ISSUED passport (a Google/GitHub account) as valid identity proof, instead of requiring you to create a BRAND-NEW bank-specific ID card from scratch. The bank never sees or stores your passport's issuing government's secret verification process (your Google password) — it just receives a signed confirmation ("yes, this person's passport is genuine") and creates its OWN internal customer record based on that confirmation.

7. Edge Cases

Case	Result
GitHub user has their email marked PRIVATE	<code>oauthUser.getAttribute("email")</code> may return <code>null</code> — must handle this fallback case explicitly (as shown above)
The SAME person tries "Sign in with Google" once and "Sign in with GitHub" another time, using the SAME email	Depending on your matching logic (matching by email), they could end up LINKED to the same local account, or create TWO separate accounts — a genuine design decision to make deliberately
Redirect URI configured in Google/GitHub's console DOESN'T EXACTLY match your app's actual callback URL	Google/GitHub REJECTS the authorization request — a very common setup mistake (even a trailing slash mismatch can cause this)

8. Common Mistakes

- Mismatched redirect/callback URIs between your provider console settings and your actual application — the single most common OAuth2 setup error.
- Not handling GitHub's nullable email attribute.
- Storing the OAuth2 provider's access token long-term instead of exchanging it for your OWN application JWT — better to establish YOUR OWN session/token model (as shown) rather than depending on the third party's token for ongoing authentication.

9. Best Practices

- After successful OAuth2 login, issue YOUR OWN JWT (as shown) rather than relying on the provider's token for your app's ongoing session — keeps your authentication model consistent regardless of which provider (or traditional username/password) was used to log in.
- Store BOTH the provider name and provider-specific user ID (not just email) in your local `user` table, to correctly distinguish/link accounts across multiple providers.

10. Frequently Asked Interview Questions

- 1 **What problem does OAuth2 solve?** Letting users log in via a trusted third party (Google, GitHub) without your application ever seeing or storing their actual third-party password.
- 2 **What are the key roles in OAuth2?** Resource Owner (the user), Client (your app), Authorization Server (Google/GitHub's login server), Resource Server (holds the protected data).
- 3 **What is an Authorization Code, and why is it exchanged server-to-server rather than used directly?** A short-lived code the browser receives after login; it's exchanged for the actual Access Token in a SERVER-to-SERVER call (using your Client Secret) so the sensitive Client Secret and final Access Token are never exposed to the browser/user directly.
- 4 **What URL pattern does Spring Security use by default for OAuth2 redirect/callback URIs?** `/login/oauth2/code/{registrationId}` (e.g., `/login/oauth2/code/google`).
- 5 **Why would you issue your OWN JWT after a successful OAuth2 login rather than just using the provider's token?** To keep a consistent authentication/session model across ALL login methods (traditional and OAuth2), independent of any particular third-party provider's token format or lifetime.

TOPIC 5: ROLE-BASED ACCESS CONTROL (RBAC)

1. Introduction

What is it? RBAC is the specific AUTHORIZATION MODEL (building on Topic 2) where permissions are assigned to defined ROLES (e.g., `ADMIN`, `USER`, `MANAGER`), and users are assigned one or more of those roles — rather than assigning permissions to individual users directly, one by one.

2. Modeling Roles in Your Entity

```
@Entity
public class User {
    @Id @GeneratedValue private Long id;
    private String username;
    private String password;

    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "user_roles", joinColumns = @JoinColumn(name = "user_id"))
    @Column(name = "role")
    private Set<String> roles = new HashSet<>(); // e.g., {"USER"}, or {"USER", "ADMIN"}
}
```

3. Enforcing Roles — URL and Method Level

```
// URL-level (in SecurityConfig)
.requestMatchers("/api/admin/**").hasRole("ADMIN")
.requestMatchers("/api/reports/**").hasAnyRole("ADMIN", "MANAGER")

// Method-level (Topic 2's @PreAuthorize, AOP-based)
@PreAuthorize("hasRole('ADMIN')")
public void deleteAllData() { }
```

Naming gotcha: `hasRole("ADMIN")` internally checks for an authority NAMED `"ROLE_ADMIN"` — Spring Security AUTOMATICALLY prepends `"ROLE_"` when using `hasRole()`. If you're granting authorities manually, make sure they're either prefixed with `ROLE_` (for `hasRole()`) or use `hasAuthority("ADMIN")` instead (no automatic prefixing) for non-role-style permissions.

4. Real Life Analogy

- **Bank:** RBAC is like the bank defining STANDARD JOB TITLES ("Teller," "Branch Manager," "Loan Officer"), each with a PRE-DEFINED set of permissions — when a new employee is hired, they're simply ASSIGNED one of these existing titles, rather than the bank individually deciding, from scratch, exactly what THIS SPECIFIC person can do.

5. Frequently Asked Interview Questions

- 1 **What is RBAC?** An authorization model where permissions are grouped into named ROLES, and users are assigned roles, rather than having permissions assigned individually per user.
- 2 **What does `hasRole("ADMIN")` actually check internally?** Whether the user has an authority named EXACTLY `"ROLE_ADMIN"` — Spring Security automatically prepends the `"ROLE_"` prefix.
- 3 **What's the difference between `hasRole()` and `hasAuthority()`?** `hasRole()` automatically prepends `"ROLE_"` to the name you provide; `hasAuthority()` checks the EXACT authority string with no automatic prefixing — useful for non-role-style, fine-grained permissions.

TOPIC 6: PASSWORD ENCODING

1. Introduction

What is it? Password Encoding is the practice of NEVER storing a user's actual password directly — instead storing a one-way, cryptographically-secure HASH of it, so even if your database is ever breached, the ORIGINAL passwords cannot be recovered.

2. Why BCrypt (Not Plain SHA-256/MD5)

Aspect	MD5/SHA-256 (general-purpose hashing)	BCrypt (password-specific hashing)
Speed	EXTREMELY fast — designed for general data integrity checks	DELIBERATELY SLOW — designed specifically to resist brute-force attacks
Salt	Not automatic — must be added manually	AUTOMATICALLY generates and embeds a unique salt PER password
Configurable "work factor"	No	Yes — can be made SLOWER over time as hardware improves, keeping brute-force attacks impractical
Recommendation for passwords	AVOID — too fast, making brute-force/rainbow-table attacks feasible	The STANDARD, correct choice

3. Code Example

```
@Configuration
public class SecurityConfig {
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(); // default strength (10) - a reasonable, tested default
    }
}

@Service
public class UserRegistrationService {
    private final PasswordEncoder passwordEncoder;
    private final UserRepository userRepository;

    public UserRegistrationService(PasswordEncoder passwordEncoder, UserRepository userRepository) {
        this.passwordEncoder = passwordEncoder;
        this.userRepository = userRepository;
    }

    public void register(String username, String rawPassword) {
        User user = new User();
        user.setUsername(username);
        user.setPassword(passwordEncoder.encode(rawPassword)); // NEVER store rawPassword directly!
        userRepository.save(user);
    }
}
```

Verification happens AUTOMATICALLY inside Spring Security's authentication process (comparing the raw login attempt against the stored hash via `passwordEncoder.matches(raw, encoded)`) — you rarely call `matches()` yourself directly.

4. Real Life Analogy

- **Bank:** Storing a plain password is like a bank keeping a customer's ACTUAL physical house key on file. BCrypt hashing is like the bank instead keeping a UNIQUE, irreversible IMPRESSION/mold of that key — useful for VERIFYING a matching key later, but USELESS to an attacker who steals the mold, since they can't reconstruct the original key from it.

5. Common Mistakes

- Storing passwords in plain text, or using a fast general-purpose hash (MD5/SHA-256) instead of a purpose-built, deliberately-slow password hashing algorithm (BCrypt/Argon2).
- Writing your own manual password comparison logic instead of using `PasswordEncoder.matches()`.

6. Frequently Asked Interview Questions

- 1 **Why shouldn't you store passwords in plain text?** A database breach would expose every user's REAL password directly, usable to compromise their accounts (and often OTHER accounts, since password reuse is common).
- 2 **Why is BCrypt preferred over MD5/SHA-256 for passwords?** BCrypt is DELIBERATELY slow and automatically salts each password uniquely, making brute-force and rainbow-table attacks impractical — MD5/SHA-256 are too FAST, designed for a different purpose (general data integrity, not password security).
- 3 **How do you verify a login attempt against a stored BCrypt hash?** Using `passwordEncoder.matches(rawPassword, storedHash)` — never manually re-implementing hash comparison yourself.

TOPIC 7: REFRESH TOKENS — FULL FLOW

1. Introduction

What is it? A Refresh Token is a LONG-LIVED, SEPARATE token issued alongside a JWT Access Token (Topic 3) — used SOLELY to obtain a NEW, short-lived Access Token once the original expires, WITHOUT forcing the user to log in again with their password.

2. Why Both an Access Token AND a Refresh Token

ACCESS TOKEN: short-lived (e.g., 15 min) - used on EVERY API request - if stolen, damage is LIMITED by short expiry
REFRESH TOKEN: long-lived (e.g., 7 days) - used ONLY to get a new Access Token - stored/transmitted MORE carefully
(often in an HttpOnly cookie, NEVER sent with every regular API request)

This gives you the SECURITY benefit of short-lived access tokens (limiting damage if leaked) WITHOUT the USABILITY cost of forcing frequent re-logins.

3. FULL STEP-BY-STEP Implementation

STEP 1 — Store Refresh Tokens (Database-Backed, So They Can Be Revoked)

```
@Entity
public class RefreshToken {
    @Id @GeneratedValue private Long id;

    @Column(unique = true, nullable = false)
    private String token;

    private String username;
    private Instant expiryDate;
}
```

STEP 2 — Generate a Refresh Token on Login

```
@Service
public class RefreshTokenService {
    private final RefreshTokenRepository repository;

    public RefreshToken createRefreshToken(String username) {
        RefreshToken refreshToken = new RefreshToken();
        refreshToken.setUsername(username);
        refreshToken.setToken(UUID.randomUUID().toString()); // a random, opaque, unguessable string
        refreshToken.setExpiryDate(Instant.now().plusSeconds(7 * 24 * 60 * 60)); // 7 days
        return repository.save(refreshToken);
    }

    public RefreshToken verifyExpiration(RefreshToken token) {
        if (token.getExpiryDate().isBefore(Instant.now())) {
            repository.delete(token);
            throw new RuntimeException("Refresh token expired. Please log in again.");
        }
        return token;
    }
}
```

STEP 3 — Return BOTH Tokens at Login

```
@PostMapping("/login")
public ResponseEntity<AuthResponse> login(@RequestBody LoginRequest request) {
    authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(request.username(), request.password()));

    String accessToken = jwtUtil.generateToken(request.username(), "USER");
    RefreshToken refreshToken = refreshTokenService.createRefreshToken(request.username());

    return ResponseEntity.ok(new AuthResponse(accessToken, refreshToken.getToken()));
}

record AuthResponse(String accessToken, String refreshToken) {}
```

STEP 4 — Build a `/refresh` Endpoint

```
@PostMapping("/refresh")
public ResponseEntity<AuthResponse> refreshAccessToken(@RequestBody RefreshRequest request) {
    RefreshToken storedToken = refreshTokenRepository.findByToken(request.refreshToken())
        .orElseThrow(() -> new RuntimeException("Invalid refresh token"));

    refreshTokenService.verifyExpiration(storedToken);

    String newAccessToken = jwtUtil.generateToken(storedToken.getUsername(), "USER");

    return ResponseEntity.ok(new AuthResponse(newAccessToken, storedToken.getToken())); // SAME refresh
    token reused
}

record RefreshRequest(String refreshToken) {}
```

STEP 5 — Client-Side Flow

1. Client logs in -> receives {accessToken, refreshToken}
2. Client uses accessToken on every API request (Authorization: Bearer ...)
3. Access token EXPIRES after 15 minutes -> API starts returning 401 Unauthorized
4. Client AUTOMATICALLY calls POST /api/auth/refresh with the STORED refreshToken
5. Server validates the refresh token, issues a BRAND-NEW accessToken
6. Client resumes normal API calls with the NEW accessToken - user experiences NO interruption!
7. This repeats until the REFRESH token itself expires (e.g., after 7 days) -> THEN, and only then, does the user need to actually log in again with their password

4. Revoking Refresh Tokens (Logout)

```
@PostMapping("/logout")
public ResponseEntity<Void> logout(@RequestBody RefreshRequest request) {
    refreshTokenRepository.deleteByToken(request.refreshToken()); // DELETE from the database - fully
    revoked
    return ResponseEntity.noContent().build();
}
```

This is EXACTLY why refresh tokens are database-backed (unlike stateless JWTs) — it gives you the ability to genuinely REVOKE a specific user's ongoing access (e.g., on logout, or "log out all devices"), something plain stateless JWTs cannot do on their own before their natural expiration.

5. Real Life Analogy

- **Restaurant:** The Access Token is like a SHORT-VALIDITY meal voucher (expires quickly). The Refresh Token is like a LONGER-VALIDITY membership card kept safely in your wallet — when your meal voucher expires, you show your membership card to get a FRESH meal voucher instantly, without needing to re-verify your full identity from scratch each time — until the membership card ITSELF eventually expires.

6. Frequently Asked Interview Questions

1 Why use both an Access Token and a Refresh Token, instead of just one long-lived token?

Short-lived access tokens limit the damage if leaked; the longer-lived refresh token allows renewing access without re-login, balancing security with user convenience.

- 2 **Why store refresh tokens in a database (unlike stateless JWT access tokens)?** To allow genuine REVOCATION (e.g., on logout) — something a purely stateless, self-contained JWT cannot support before its natural expiration.
 - 3 **What happens when a refresh token itself expires?** The user must log in again with their actual credentials — the refresh mechanism can no longer silently renew access.
-

TOPIC 8: CORS (CROSS-ORIGIN RESOURCE SHARING) — FULL CONFIGURATION

1. Introduction

What is it? CORS is a BROWSER security mechanism that BLOCKS a web page from making requests to a DIFFERENT origin (domain/port/protocol) than the one it was loaded from, UNLESS that other origin EXPLICITLY permits it via special response headers — directly relevant whenever your frontend (e.g., `http://localhost:3000`, a React app) and backend (e.g., `http://localhost:8080`, your Spring Boot API) run on DIFFERENT origins.

2. Why the Browser Blocks This by Default (The "Same-Origin Policy")

Without CORS restrictions, a malicious website could silently make requests to OTHER sites you're logged into (your bank, your email) using YOUR browser's existing session cookies, and read the responses — a serious security risk. The Same-Origin Policy prevents this by default; CORS is the CONTROLLED, EXPLICIT exception mechanism a server uses to say "these SPECIFIC other origins are allowed to talk to me."

3. FULL STEP-BY-STEP Configuration

STEP 1 — Configure CORS Globally in SecurityConfig

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .authorizeHttpRequests(auth -> auth.anyRequest().authenticated());
        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(List.of("http://localhost:3000", "https://myapp.com")); // EXACT allowed origins
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        config.setAllowedHeaders(List.of("Authorization", "Content-Type"));
        config.setAllowCredentials(true); // needed if sending cookies/Authorization headers

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config); // applies to ALL endpoints
        return source;
    }
}
```

STEP 2 — (Alternative) Simple, Controller-Level CORS (For Quick/Small Projects)

```
@RestController
@CrossOrigin(origins = "http://localhost:3000") // simpler, but LESS flexible than the global config above
public class EmployeeController { }
```

4. The CORS Preflight Request — Internal Working

For "non-simple" requests (e.g., using PUT/DELETE, custom headers like Authorization), the browser AUTOMATICALLY sends an OPTIONS "preflight" request FIRST, BEFORE the actual request:

```
Browser -> OPTIONS /api/employees (asking: "am I ALLOWED to make a real PUT request here?")
Headers: Origin: http://localhost:3000, Access-Control-Request-Method: PUT
Server -> 200 OK
Headers: Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
```

ONLY IF this preflight succeeds does the browser THEN send the ACTUAL PUT request.

5. Real Life Analogy

- **Bank:** CORS is like a bank REFUSING to accept instructions relayed through an UNVERIFIED third party by default, UNLESS that specific third party has been EXPLICITLY pre-approved and listed on an authorized-intermediaries list — protecting customers from having their bank instructions hijacked/executed by an untrusted, unrelated source.

6. Edge Cases

Case	Result
<code>allowedOrigins("*")</code> combined with <code>allowCredentials(true)</code>	NOT ALLOWED by the CORS specification — using "*" (wildcard) for origins is INCOMPATIBLE with allowing credentials; you must list EXACT origins
Frontend and backend on the SAME origin (e.g., serving a built React app directly FROM Spring Boot)	CORS doesn't apply at all — it's ONLY relevant for genuinely cross-origin requests

7. Common Mistakes

- Using `allowedOrigins("*")` in production (or combining it incorrectly with `allowCredentials(true)`, which the spec forbids) — always list SPECIFIC, exact allowed origins in production.
- Configuring CORS at the CONTROLLER level (`@CrossOrigin`) inconsistently across different controllers, instead of one centralized, consistent global configuration.

8. Frequently Asked Interview Questions

- What is CORS?** A browser security mechanism blocking cross-origin requests by default, unless the target server explicitly permits specific origins via response headers.
- Why does the Same-Origin Policy exist?** To prevent a malicious website from silently making requests to OTHER sites using the victim's existing browser session/cookies and reading the responses.
- What is a CORS "preflight" request?** An automatic `OPTIONS` request the browser sends BEFORE certain "non-simple" requests (e.g., using `PUT/DELETE` or custom headers), checking whether the actual request would be permitted.
- Can you combine `allowedOrigins("*")` with `allowCredentials(true)`?** No — this combination is explicitly disallowed by the CORS specification; you must list specific, exact origins when credentials are involved.

TOPIC 9: CSRF (CROSS-SITE REQUEST FORGERY)

1. Introduction

What is it? CSRF is an ATTACK where a malicious website tricks a user's browser into making an UNWANTED, authenticated request to ANOTHER site the user is currently logged into (e.g., their bank) — exploiting the fact that browsers automatically attach existing session COOKIES to requests, regardless of which page/site actually triggered them.

2. How the Attack Works

1. User logs into their-bank.com (browser now holds a valid session cookie for their-bank.com)
2. User (while STILL logged in) visits a malicious site, evil-site.com
3. evil-site.com's page contains a HIDDEN form auto-submitting to:
their-bank.com/transfer?amount=10000&to=attacker-account
4. The BROWSER automatically attaches the user's EXISTING their-bank.com session cookie to this request (since cookies are sent based on the TARGET domain, not which page triggered the request) - the bank's server sees what LOOKS LIKE a legitimate, authenticated request from the actual logged-in user!

3. How Spring Security's CSRF Protection Works

Server generates a UNIQUE, UNPREDICTABLE CSRF TOKEN per user session, and requires it to be included in any STATE-CHANGING request (POST/PUT/DELETE) - typically as a HIDDEN FORM FIELD (for traditional server-rendered apps) or a custom HEADER (for JS-driven SPAs) - NOT something a malicious third-party site could know or guess, since it never has legitimate access to read cookies/tokens from the bank's own domain.

4. Code Example — CSRF in a Traditional Server-Rendered App (Enabled, Default Behavior)

```
// CSRF protection is ENABLED BY DEFAULT for stateful (session-based) Spring Security apps - no config needed!
// In a Thymeleaf template, the CSRF token is automatically included:
<form method="post" action="/transfer">
<input type="hidden" th:name="${_csrf.parameterName}" th:value="${_csrf.token}"/>
...
</form>
```

5. Why JWT-Based Stateless APIs Typically DISABLE CSRF Protection

```
http.csrf(csrf -> csrf.disable()) // seen in the JWT SecurityConfig example, Topic 3!
```

CSRF protection specifically defends against attacks exploiting AUTOMATICALLY-ATTACHED COOKIES. A properly-implemented, STATELESS JWT API doesn't rely on cookies for authentication at all — the JWT is manually attached by the CLIENT's own JavaScript code (in an `Authorization` header), which a malicious third-party site CANNOT do on the victim's behalf (it has no way to read/access the JWT stored in the legitimate app's own storage). This is WHY disabling CSRF is a standard, safe practice specifically for token-header-based stateless APIs — but NOT for traditional cookie/session-based applications.

6. Real Life Analogy

- **Bank:** CSRF protection is like a bank requiring a UNIQUE, freshly-issued authorization code (printed ONLY on the bank's own official website, never accessible to an outside party) alongside every money-transfer REQUEST — a malicious third party tricking your browser into submitting a transfer request has NO WAY to also know/include that unique, freshly-issued code, so the bank rejects the forged request.

7. Edge Cases

Case	Result
CSRF enabled (default) but a state-changing request submitted WITHOUT the token	Rejected with HTTP 403 Forbidden
CSRF disabled on a TRADITIONAL, cookie/session-based application	Genuinely DANGEROUS — reopens the exact vulnerability described in Section 2
CSRF disabled on a properly stateless, JWT-header-based API	Generally SAFE and standard practice, since the attack vector CSRF defends against doesn't apply

8. Common Mistakes

- Disabling CSRF protection on a TRADITIONAL, cookie/session-based application "just to make testing easier" — reopens a genuine, serious vulnerability.
- Assuming CSRF and CORS are the same concept — they address DIFFERENT problems (CSRF = forged requests exploiting automatic cookie attachment; CORS = controlling which origins can make cross-origin requests at all).

9. Frequently Asked Interview Questions

- 1 **What is a CSRF attack?** A malicious site tricking a user's browser into making an unwanted, authenticated request to another site the user is currently logged into, exploiting automatically-attached session cookies.
- 2 **How does Spring Security's CSRF protection defend against this?** By requiring a unique, unpredictable, session-specific CSRF token on every state-changing request — a token a malicious third-party site has no way to know or include.
- 3 **Why do stateless, JWT-header-based REST APIs typically disable CSRF protection?** Because they don't rely on automatically-attached cookies for authentication — the JWT must be manually attached by the legitimate client's own code, which a malicious site cannot do on the victim's behalf, making the CSRF attack vector inapplicable.
- 4 **Is disabling CSRF ever appropriate for a traditional, cookie/session-based application?** No — that would reopen a genuine, serious vulnerability; disabling CSRF is only appropriate for properly stateless, non-cookie-based authentication schemes.

10. Revision Notes (Entire Spring Security Section Consolidated)

SPRING SECURITY — CHEAT SHEET

=====

AUTHENTICATION: "WHO are you?" - Security Filter Chain -> AuthenticationManager -> AuthenticationProvider -> SecurityContext populated with Authentication object
AUTHORIZATION: "WHAT can you do?" - URL-based (authorizeHttpRequests) or method-based (@PreAuthorize/@PostAuthorize, built on Spring AOP)

JWT: Header.Payload.Signature - payload is ENCODED not encrypted, never put secrets in it!
Full flow: JwtUtil (generate/validate) -> JwtAuthenticationFilter (OncePerRequestFilter, extracts+validates token, sets SecurityContext) -> UserDetailsService -> login endpoint generates the token -> client sends "Authorization: Bearer <token>" on every request

OAUTH2 ("Sign in with Google/GitHub"): register app on provider console -> get client id/secret -> configure application.yml -> Spring's default redirect: /login/oauth2/code/{registrationId} -> AuthenticationSuccessHandler creates/finds local user, issues YOUR OWN JWT -> frontend link: /oauth2/authorization/{registrationId}

RBAC: hasRole("X") checks for "ROLE_X" authority (auto-prefixed); hasAuthority("X") checks the EXACT string, no prefixing

PASSWORD ENCODING: BCryptPasswordEncoder - DELIBERATELY slow, auto-salted, NEVER MD5/SHA-256 for passwords

REFRESH TOKENS: short-lived JWT access token + long-lived, DATABASE-BACKED (revocable!) refresh token -> /refresh endpoint issues a new access token without re-login

CORS: browser Same-Origin Policy exception mechanism - configure EXACT allowedOrigins (never "*" with allowCredentials(true)), preflight OPTIONS requests for non-simple calls

CSRF: defends against forged requests exploiting AUTO-ATTACHED cookies - enabled by DEFAULT for session/cookie apps, typically DISABLED for stateless JWT-header APIs (different attack surface entirely)

11. Homework (Covers Entire Spring Security Section)

- 1 Implement the full JWT flow from Topic 3 end-to-end: register a user (BCrypt-hashed password), log in to get a token, and access a protected endpoint using that token.
- 2 Set up "Sign in with Google" following Topic 4's exact steps against a real Google Cloud project, and verify a new local user is created on first login.
- 3 Add refresh token support to your JWT implementation, and test that an expired access token can be renewed via /refresh without re-entering credentials.
- 4 Configure CORS to allow a frontend running on a different port, and use your browser's Network tab to observe an actual CORS preflight OPTIONS request.

FINAL SUMMARY — SPRING SECURITY COMPLETE

This concludes the **Spring Security** section — 9 topics, with full, practical, step-by-step integration guides as requested:

- 1 **Authentication** — the Security Filter Chain, "WHO are you."
- 2 **Authorization** — URL and method-level (@PreAuthorize), "WHAT can you do."

- 3 **JWT** — a complete, working, step-by-step stateless authentication implementation.
- 4 **OAuth2** — full Google Cloud Console AND GitHub OAuth App setup steps, with working Spring Boot integration code for both.
- 5 **Role-Based Access Control** — `hasRole()` vs `hasAuthority()`.
- 6 **Password Encoding** — BCrypt, and why NOT MD5/SHA-256.
- 7 **Refresh Tokens** — the full short-lived-access + long-lived-revocable-refresh flow.
- 8 **CORS** — full configuration, the preflight mechanism.
- 9 **CSRF** — the attack, the defense, and why JWT APIs typically disable it.

You now have complete, hands-on knowledge to secure a real Spring Boot application end-to-end — from traditional login through modern JWT and social sign-in. Next: REST APIs, formalizing the architectural principles (HTTP methods, status codes, versioning) underlying everything built across the Spring Boot and Spring Security sections.

(END OF SPRING SECURITY MASTER NOTES — ready to proceed to REST APIs whenever you say "Next Topic.")